

ROTEIRO DE APRESENTAÇÃO

Ceres Diagnóstico — Defesa de Artigo PSI

Namem Rachid Jaudy Neto · IFMT Cuiabá · Junho 2026

Duração total	30 minutos (mais 10 min Q&A)
Slides	23 slides
Ritmo médio	~1 min 18 s / slide
Vídeo	demo_ceres_1.5x.mp4 (slide 20)

■ Fala normal

★ Ênfase / ponto-chave

■ Alerta / limitação

✓ Resultado positivo

→ Transição

SLIDE 1 Capa

0:00–0:30

■ Bom dia. Meu nome é Namem, e vou apresentar o Ceres Diagnóstico — um sistema TinyML embarcado para detecção de doenças em tomateiro.

■ O trabalho integra três frentes: um microcontrolador ESP32-S3, um backend Django e um app Flutter. Tudo conectado por MQTT.

→ *Vou começar pelo problema que motivou o projeto.*

SLIDE 2 Roteiro

0:30–0:45

■ Mostrarei o problema, como construí o modelo, a surpresa que foi o gap campo-laboratório, os cinco experimentos, as três arquiteturas de inferência e o sistema em funcionamento.

→ *Partindo do problema.*

SLIDE 3 Problema e Motivação

0:45–2:00

■ O tomateiro é a terceira maior produção mundial de vegetais — R\$ 10 bilhões por ano só no Brasil.

■ Doenças foliares podem causar perda total de safra. E diagnóstico preciso depende de agrônomos especializados que simplesmente não chegam ao pequeno produtor rural.

★ **Sorriso-MT concentra grande parte dos produtores de tomate do MT. A maioria são pequenos agricultores sem assistência técnica regular.**

■ Conectividade rural é instável. Uma solução 100% cloud não funciona no campo.

→ *Nossa proposta resolve exatamente esse cenário.*

SLIDE 4 Nossa Proposta

2:00–3:00

■ Um sistema de baixo custo que classifica 10 doenças do tomateiro e funciona offline — sem internet.

■ O modelo cabe em 638 KB, roda em microcontrolador, e o código é totalmente aberto.

→ *Para construir isso, precisávamos de dados. Vou mostrar o pipeline.*

SLIDE 5 Objetivos e Contribuições

3:00–4:00

- O objetivo é integrar TinyML, IoT e app móvel num único pipeline reproduzível.
- As cinco contribuições: dez classes com pipeline aberto; ESP32-S3 validado a 692 ms; três arquiteturas de inferência; validação em três datasets independentes; e cinco experimentos documentados, incluindo os negativos.
- ★ **Resultado negativo documentado é contribuição — é o que evita que outros repitam o mesmo caminho.**

→ *Começando pelo dataset.*

SLIDE 6 Dataset e Pipeline

4:00–5:00

- Usamos o PlantVillage — 18.160 imagens de folha de tomate, 10 classes, licença CC BY 4.0.
- Split 70/15/15 com seed fixo para reproduzibilidade. Augmentation offline multiplicou o treino por 6, chegando a 88.949 imagens. O test set de 2.734 imagens nunca foi visto durante o treinamento.

→ *Vamos ver como são essas dez classes.*

SLIDE 7 As 10 Classes

5:00–5:30

- Estas são as dez classes reais do modelo — fotos do PlantVillage. Notem a variabilidade: requeima tem lesão úmida escura, septoriose tem pontos brancos com halo amarelo, pinta-preta tem manchas concêntricas.
- É essa diversidade visual que torna o problema desafiador.

→ *Antes de treinar, precisei escolher a arquitetura. Próximo slide.*

SLIDE 8 Comparativo de Modelos

5:30–6:30

- Esta tabela está no artigo. Cinco arquiteturas avaliadas para rodar no ESP32-S3.
- ResNet-50 e EfficientNet-B0: grandes demais para o microcontrolador — sem suporte no ESP32-S3. YOLO foi descartado imediatamente — é um detector de objetos com bounding box, incompatível com classificação de folha única. Tamanho mínimo ~6 MB, dez vezes maior que o nosso.
- ★ **MobileNetV2: único com suporte comprovado no ESP32-S3, arena de 200 KB dentro dos 512 KB disponíveis, acurácia superior ao MobileNetV1. Escolha evidente.**

→ *Com a arquitetura definida, vamos ao treinamento.*

SLIDE 9 Treinamento e Quantização

6:30–9:00

- Treinamento em duas fases — transfer learning correto. Fase 1: backbone ImageNet congelado, dez épocas. Fase 2: fine-tuning das últimas 30 camadas, quarenta épocas. Melhor val_acc: 97,90% na época 46.
- Os três números contam a história da quantização: 62% com INT8 sem calibração — queda de 30 pp. Isso é o que acontece no Edge Impulse automático.
- No TensorFlow local, com 50 batches do validation set como representative_dataset, a perda caiu de 30,5 pp para apenas 2,37 pp. O modelo embarcado entrega 95,76% — não os 98,13% do float.

■ *Esse delta de 2,37 pp é o custo da quantização bem feita. Vale o ganho: 4× menos espaço, 3× menos latência.*

→ Ótimo resultado em laboratório. O problema veio quando testamos em campo.

SLIDE 10

Matriz de Confusão

9:00–10:00

■ 2.618 de 2.734 corretas no test set de laboratório. As classes difíceis são as com lesões necróticas escuras — pinta-preta, septorrose e mancha bacteriana tendem a se confundir entre si.

■ Classes quase perfeitas: vira-cabeça e saudável — padrões visuais bem distintos.

■ *Esse padrão de confusão persistiu em todos os experimentos. É uma limitação do modelo atual.*

→ *E em campo real?*

SLIDE 11

Gap Laboratório–Campo

10:00–12:00

■ Testamos com o PlantDoc — 1.353 imagens capturadas no campo por vários fotógrafos, em condições reais de iluminação, ângulo e fundo.

★ **95,76% em laboratório. 20,77% em campo. Queda de 75 pp.**

■ Mas não estamos sozinhos. Mohanty 2016 publicou 99% no PlantVillage. Singh 2020 avaliou o mesmo tipo de modelo no PlantDoc e obteve ~31%. Xu 2024 revisou 42 trabalhos e documentou queda entre 29 e 58 pp.

■ *Esse fenômeno tem nome: domain shift. O modelo aprende o fundo cinza controlado do laboratório, não as features diagnósticas da lesão.*

→ *Então abrimos uma investigação com cinco experimentos para tentar resolver.*

SLIDE 12

Os 5 Experimentos

12:00–14:30

■ Exp A: Edge Impulse sem calibração INT8. 62% — confirmou o problema da quantização.

■ Exp B: TF local com calibração. 95,76% em laboratório, 20,77% em campo. Esse é o modelo base.

■ *Exp C: Tentamos trocar o fundo com augmentation sintética — rembg, U2-Net, 177.698 composições. Resultado: 20,24% em campo. Pior do que o Exp B. Resultado negativo documentado.*

■ Exp D: Fine-tuning com 677 imagens reais do PlantDoc. Subiu para 30,43% em campo — +10 pp.

✓ **Exp E: Adicionamos Focal Loss $\gamma=2$, que reduz o peso das imagens fáceis de laboratório. +16 pp no Tomato-Village indiano. É o modelo final.**

→ *Vou detalhar o resultado negativo — é pedagógico.*

SLIDE 13

Exp C — Resultado Negativo

14:30–16:00

■ A hipótese era razoável: se o modelo aprende o fundo, trocar o fundo deveria forçá-lo a olhar a lesão.

■ Geramos 177.698 composições sintéticas com rembg. O resultado foi 20,24% — marginalmente pior que o baseline.

■ O que aconteceu: artefatos de borda do rembg e iluminação inconsistente criaram um domínio sintético que não representa o campo real. A classe saudável caiu a 0% em campo.

★ Lição: síntese não substitui dado de campo. Só fine-tuning com imagens reais foi efetivo.

→ O que funcionou — Exp D e E.

SLIDE 14

Exp D e E — O Que Funcionou

16:00–17:30

■ Exp D: 677 imagens reais do PlantDoc/train, repetidas 10 vezes, com as 69 de teste nunca vistas. +10 pp.

■ O fator limitante identificado: volume de dados de campo, não o método de treinamento.

✓ Exp E: Focal Loss $\gamma=2$ força o modelo a focar nas features da lesão, reduzindo o peso das imagens fáceis de fundo cinza. +16 pp no Tomato-Village.

→ Mas esse ganho não é uniforme geograficamente.

SLIDE 15

Gap Geográfico e Colapso de Classe

17:30–18:30

■ A acurácia em campo cai com a distância geográfica do PlantDoc: EUA/Europa 30,43%, Índia 27,65%, Bangladesh 18,13%.

■ Barbedo 2019 previu isso: variedades locais, iluminação tropical e estágio fenológico distinto degradam o desempenho.

■ No Tomato-Village, 73% das folhas saudáveis indianas foram classificadas como septoriose — colapso de classe clássico sob domain shift extremo. O Exp E mitigou parcialmente.

→ Com o modelo definido, a questão passou a ser: onde rodar?

SLIDE 16

Modelo Pronto — Onde Rodar?

18:30–20:30

■ O mesmo modelo INT8 de 638 KB pode rodar em três caminhos: direto no ESP32-S3, no smartphone Android, ou na nuvem via Django.

■ Caminho ①: ESP32-S3, 692 ms, offline, MQTT/TLS.

■ Caminho ②: Android on-device, tflite_flutter. Offline, sem hardware adicional.

■ Caminho ③: Cloud Django, 306 ms, HTTPS, Railway + PostgreSQL. Online, pipeline completo validado.

★ Pivô de engenharia: o plano original era ESP32-S3 com câmera OV5640, 100% autônomo. Ao integrar, percebemos que a câmera precisaria de Sprint 2 para calibração. A adaptação foi oferecer o smartphone como sensor — mesmo modelo, mais flexibilidade imediata.

→ Comparando os três caminhos.

SLIDE 17

Comparativo Edge / Mobile / Cloud

20:30–21:30

■ Edge: 692 ms determinístico, offline, ESP32-S3 ~R\$ 80. Mobile: ~200–400 ms estimados — não medimos empiricamente neste ciclo, limitação documentada — app no celular do produtor. Cloud: 306 ms via HTTPS, Railway.

■ Os três estão em produção ou validados. O mesmo arquivo INT8 de 638 KB nos três caminhos.

→ [Agora deixa eu mostrar o que de fato construímos.](#)

SLIDE 18

O Que Construímos

21:30–22:30

■ Este slide conta a história da construção. Começamos com o plano: ESP32-S3 autônomo com câmera OV5640. Durante a integração, percebemos que a câmera demandaria Sprint 2 completa para calibração de buffer e exposição.

■ A decisão: transformar o ESP32 em sensor IoT de ambientação — temperatura, umidade do ar, umidade do solo, publicados via MQTT. O smartphone assume o papel de câmera inteligente, com o mesmo modelo INT8 embarcado via tflite_flutter.

★ **ceres_mobilenetv2_int8.tflite · 638 KB · tflite_flutter 0.12.1 · mesmo arquivo do ESP32-S3 · inferência offline ~200–400 ms.**

■ Essa não foi uma limitação — foi uma decisão de engenharia. O sistema ficou mais flexível e mais acessível ao produtor.

→ [Vamos ver o sistema funcionando.](#)

SLIDE 19

O Sistema em Funcionamento

22:30–23:30

■ App Flutter: tela IoT mostrando 29,5 °C, 49% umidade do ar, 34% umidade do solo, status ONLINE.

■ Hardware real: ESP32-S3 com DHT22 e sensor capacitivo de solo funcionando.

■ Pipeline completo: ESP32 publica via MQTT QoS 1 → HiveMQ Cloud → Django persiste com timestamp e GPS → Flutter sincroniza offline/online com Drift SQLite.

→ [Vou mostrar o vídeo demonstrativo.](#)

SLIDE 20

Demonstração

23:30–25:30

■ Vídeo a 1,5× — dura cerca de 1 minuto. Mostra: captura de folha pela câmera do app, diagnóstico nos três caminhos, publicação MQTT do ESP32, histórico com GPS no mapa.

■ *Se o vídeo não abrir: abra [demo_ceres_1.5x.mp4](#) diretamente no player.*

→ [Conclusão.](#)

SLIDE 21

Conclusão

25:30–27:30

★ **Manchete: TinyML agrícola funciona — o gargalo é o dado, não o hardware.**

✓ **Card 1 — Viabilidade: MobileNetV2 INT8 roda no ESP32-S3 em 638 KB e 692 ms. Calibração INT8 reduziu a queda de quantização de 30,5 pp para 2,37 pp. Pipeline completo: ESP32-S3, Android,**

Django — integrados.

■ Card 2 — Gap lab-campo: 95,76% em laboratório, 18–30% em campo. Aug. sintética falhou. Fine-tuning real: +10 pp. Focal Loss: +16 pp. Gap persiste — o método está identificado, o dado ainda falta.

■ Card 3 — O gargalo real: não é o hardware — é o dado. Não existe dataset brasileiro de campo com volume e diversidade suficientes para generalização real. Esse é o próximo passo da pesquisa.

→ [Trabalhos futuros.](#)

SLIDE 22

Trabalhos Futuros

27:30–29:00

■ Quatro próximos passos: dataset de campo brasileiro com produtores locais — é o fator que mais limita a generalização. Integração da câmera OV5640, fechando o ciclo embarcado autônomo. Validação com produtores de Sorriso-MT. Expansão para outras culturas.

→ [Para encerrar.](#)

SLIDE 23

Obrigado / Perguntas

29:00–30:00

■ O Ceres Diagnóstico demonstra que TinyML embarcado é viável para diagnóstico agrícola de baixo custo. O gap campo-laboratório é o principal desafio aberto — e o caminho para resolvê-lo está identificado: dados reais de campo.

★ Código e modelos: github.com/Namem/extensao2

■ Obrigado. Fico à disposição para perguntas.

PERGUNTAS FREQUENTES (Q&A;)

Q: Por que não usar YOLO?

R: YOLO é um detector de objetos — retorna bounding boxes, não classes de classificação. Incompatível com nossa tarefa de classificar a folha inteira. Além disso, o menor YOLO ocupa ~6 MB, 10× maior que nosso modelo de 638 KB.

Q: Por que MobileNetV2 e não ResNet ou EfficientNet?

R: MobileNetV2 foi projetado para dispositivos móveis e MCUs: depthwise separable convolutions reduzem FLOPs em ~8–9×. ResNet50 tem ~25 MB de parâmetros. EfficientNet-Lite requer arenas maiores e não havia suporte estável para TFLite Micro INT8 na época dos experimentos.

Q: 20–30% em campo é aceitável?

R: É um resultado honesto para um primeiro ciclo de pesquisa. A literatura mostra queda similar. O caminho foi identificado: fine-tuning com dados reais (Exp D/E). O próximo passo é coletar imagens no campo com produtores de Sorriso-MT.

Q: Por que a câmera OV5640 não foi integrada neste ciclo?

R: Integração da câmera requer calibração de buffer de captura, correção de exposição e latência de leitura — trabalho de Sprint 2. A decisão foi isolar a latência da CNN pura (692 ms) carregando imagens como array C. O smartphone serve como sensor alternativo no caminho ②.

Q: 692 ms é rápido o suficiente para uso em campo?

R: Sim. O produtor leva 2–5 segundos para posicionar a folha adequadamente. 692 ms é imperceptível. A meta de 300 ms é para futura otimização com OV5640 pipeline completo, não um bloqueador atual.

Q: Como garantir privacidade dos dados do produtor?

R: No caminho Edge (ESP32-S3) e no caminho Mobile (Android), a imagem nunca sai do dispositivo — apenas o resultado da classificação em texto é transmitido. No caminho Cloud, a imagem trafega via HTTPS para o servidor Django. O produtor escolhe o caminho.

Q: O sistema funciona sem internet?

R: Os caminhos Edge e Mobile funcionam 100% offline. O histórico é sincronizado quando o usuário reconecta via Drift SQLite. O caminho Cloud requer conectividade.

Q: Como o sistema será validado com produtores reais?

R: Sprint 3 prevê coleta de imagens com pequenos produtores de Sorriso-MT, validação de usabilidade e teste de campo. Esse dataset brasileiro de campo é o principal trabalho futuro.
